KALAVAI: Predicting When Independent Specialist Fusion Works
A Quantitative Model for Post-Hoc Cooperative LLM Training

Anonymous Author(s)
Affiliation
Address
email

Abstract

When multiple contributors independently fine-tune a shared language model checkpoint on different domains and their specialists are fused post-hoc via a lightweight learned router, the resulting model consistently outperforms any individual specialist across all domains simultaneously. The key intuition is that specialists trained on different domains develop complementary internal representations; a learned router can exploit this structure at inference without any specialist retraining. While prior work shows post-hoc fusion of independently trained models is feasible, this paper characterises when it succeeds, how much gain to expect, and what coordination it requires.

We introduce KALAVAI[1], a protocol in which contributors train on private data without exchanging gradients, activations, or model updates, then submit checkpoints for lightweight Mixture-of-Experts (MoE) routing (500 router training steps on mixed data). Under this protocol, KALAVAI improves over the best individual specialist by $+7.70\%$, $+7.49\%$, and $+6.53\%$ across Pythia-410M, 1B, and 6.9B respectively (3-seed means). Fusion gain is predictable from specialist divergence: gain $\approx 0.83 \times$ divergence $- 2.80$ ($R^2 = 0.872$, n=8 conditions, 3–26% divergence range). While derived from a limited number of conditions, this relationship suggests divergence as a practical screening criterion prior to full cooperative training. Notably, conversion efficiency—the ratio of fusion gain to specialist divergence—increases with model scale ($0.75\times$ at 6.9B vs. $0.49\times$ at smaller scales), suggesting the cooperative mechanism strengthens rather than degrades as models become more capable. In high-divergence settings gains are substantially larger: cross-lingual fusion of Tamil, Yoruba, Welsh, and code specialists achieves $+21.87\%$ ($\pm$0.12pp, 3-seed mean), reducing Yoruba perplexity from 41.9 to 7.7.

Three structural conditions govern the protocol. *Shared initialisation* is necessary: mismatched checkpoints degrade routing coherence, because the representational geometry that enables clean routing is established at initialisation, not during training. *Layer freezing* is beneficial only beyond approximately 10,000 specialist training steps ($+2.4$pp advantage at 20,000 steps; unnecessary below that threshold). *Routing must be learned*: a trained linear router approaches the domain-level oracle—a hypothetical router with perfect per-token domain knowledge—with a gap of $<10^{-5}$ nats (natural units of entropy; 1 nat $\approx$ 1.44 bits) at 410M scale, while uniform weight averaging reduces performance by 1.2% relative to the best individual specialist. Section 3 formalises the protocol; Sections 4.2–4.8 characterise these conditions empirically; Appendix S details the evaluation protocol.

[1] KALAVAI (*kalaVAI*) is the ISO 15919 romanisation of the Tamil word for "fusion" or "mixing".

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

1 Introduction

Training competitive language models requires centralised compute at a scale most researchers and institutions cannot access. Frontier training runs demand hundreds of synchronised GPUs for weeks, concentrating capability in organisations with sufficient infrastructure. This creates a practical barrier: strong models are currently accessible primarily to those who can build or fund them. This raises a natural question: can the training effort itself be distributed, so that many contributors—each with their own data, hardware, and constraints—collectively produce a model none of them could build alone? We demonstrate that the answer is yes: language model training can be decentralised into independent specialist contributions, with predictable and quantifiable gains achieved through post-hoc fusion.

Cooperative specialist fusion. A structurally different approach is possible. If multiple contributors each fine-tune an independent copy of a shared base checkpoint on their own domain, the resulting specialists collectively aggregate complementary knowledge that no single contributor could build alone.

The key insight is that specialists trained on different domains diverge in *complementary* directions from the shared initialisation point: each becomes highly competent on its own domain while degrading on out-of-domain data. A lightweight router—trained on a small mixed-domain dataset after all specialists have submitted their checkpoints—exploits this structure by assigning each input token to its most relevant specialist, recovering specialisation quality across all domains simultaneously. This requires *no communication during training*: contributors work independently, asynchronously, on their own hardware, using their own private data. The only coordination is the shared initial checkpoint.

The branch-train-mix (BTX) paradigm [Sukhbaatar et al., 2024] demonstrates that MoE fusion of independently trained models is feasible. However, the conditions under which post-hoc specialist fusion succeeds—what level of divergence is required, how training duration interacts with layer freezing, what routing architecture is necessary—have not been empirically characterised. This paper provides that characterisation, along with a protocol that operationalises cooperative training under realistic privacy constraints.

The KALAVAI protocol. KALAVAI operationalises cooperative specialist fusion as a four-step protocol: (1) a coordinator distributes a shared base checkpoint; (2) each contributor fine-tunes independently on their domain; (3) contributors submit their checkpoints; (4) a lightweight router is trained on a small mixed-domain dataset and used for inference. Contributors never share gradients, intermediate activations, or data. The only shared artefact is the initial checkpoint.

Contributions. We report five empirical findings:

1. **A predictive divergence–gain relationship** (Section 4.8). Fusion gain is predictable from specialist divergence before training begins: gain $\approx 0.83 \times$ divergence $- 2.80$ ($R^2 = 0.872$, n=8 conditions, 3–26% divergence range). Intuitively, more divergent specialists encode more complementary knowledge, giving the router more to exploit. In practice, this means practitioners can screen specialists by divergence on a small validation set before committing to full cooperative training.[2]
2. **Near-oracle routing with a linear router** (Section 5). A trained linear router approaches the domain-level oracle (perfect per-token domain assignment) with a gap of $<10^{-5}$ nats at 410M scale. Surprisingly, routing architecture does not matter—linear and MLP routers perform equivalently—but routing must be trained; uniform weight averaging degrades performance below the best individual specialist. The bottleneck is whether the router is trained at all, not its architectural capacity.
3. **Consistent gains across model scales** (Section 4.2). KALAVAI improves over the best specialist by $+7.70\%$ at 410M, $+7.49\%$ at 1B, and $+6.53\%$ at 6.9B (3-seed means). Reduced gain at 6.9B reflects lower specialist divergence at that scale, not protocol degradation; conversion efficiency is in fact higher at 6.9B ($0.75\times$) than at smaller scales ($0.49\times$). This suggests the mechanism strengthens, not weakens, as models grow more capable.

[2] All results use the per-domain equal-weight protocol; see Appendix S.

4. **A training-duration crossover for layer freezing** (Section 4.4). Layer freezing is unnecessary below approximately 10,000 specialist training steps; above that threshold, freezing lower layers preserves routing coherence and provides a measurable performance advantage ($+2.4$pp at 20,000 steps). Intuitively, short specialisation runs naturally retain compatible lower-level features; only extended training erodes this shared geometry, at which point frozen layers act as a structural anchor.
5. **Substantial gains in high-divergence settings** (Section 4.7). Cross-lingual cooperative fusion (Tamil, Yoruba, Welsh, and code) achieves $+21.87\%$ ($\pm$0.12pp, 3-seed mean); an 18-specialist multilingual federation achieves $+21.13\%$ ($\pm$0.01pp, 3 seeds). In practice, gains are largest precisely where the base model is least competent: Yoruba perplexity falls from 41.9 to 7.7 without any shared data between language communities.

2 Related Work

Branch-Train-Mix and MoE merging. The closest prior work is BTX [Sukhbaatar et al., 2024], which demonstrates MoE fusion of independently branched models. KALAVAI extends BTX with empirical characterisation of the conditions under which fusion succeeds: the training-duration crossover (Section 4.4), the routing learning requirement, near-oracle routing performance, and a compute-matched monolithic baseline. The MoErging survey [Yadav et al., 2024] taxonomises routing-based merging; PHATGOOSE [Muqeeth et al., 2024] achieves +11% zero-shot gain via learned routing among fine-tuned models. Pari [2025] provides a CKA-based theoretical account of why weight averaging fails; KALAVAI provides the empirical complement (+7.72% MoE vs. -3.4% weight averaging vs. best specialist).

Weight interpolation and distillation. Model soups [Wortsman et al., 2022], TIES-Merging [Yadav et al., 2023], and DARE [Yu et al., 2024] merge via weight interpolation; FuseLLM [Wan et al., 2024] uses knowledge distillation. Both families preserve a single model; KALAVAI retains all specialist parameters and routes at inference. Sparse Upcycling [Komatsuzaki et al., 2023] and STAR [Qin et al., 2025] initialise or compose MoE models from dense checkpoints with joint or modular training; KALAVAI specialists train with *zero communication*.

Federated learning. Federated approaches [McMahan et al., 2017] distribute training with periodic gradient synchronisation. KALAVAI requires zero communication during training and is fully asynchronous; contributors are coordinated only by the shared initial checkpoint.

3 Method

The KALAVAI protocol consists of four phases.

Phase 1: Shared initialisation. A coordinator selects a publicly available base checkpoint θ_0 and distributes it to all contributors. All specialists begin from *identical* weights. This shared initialisation is the core structural guarantee that enables post-hoc fusion: specialists diverge in representation space, but their representational geometry remains compatible because they begin from the same point. Hash verification ensures all contributors use exactly the same checkpoint. This establishes the causal mechanism: shared initialisation preserves a common representational geometry; independent specialisation then induces structured divergence via catastrophic forgetting on out-of-domain tokens; trained routing exploits this divergence to recover domain-specific competence across all specialists simultaneously.

Phase 2: Layer freezing (optional). The first K transformer layers may be frozen during specialist training. Frozen layers ensure that lower-level representations remain shared across all specialists—a structural anchor that prevents over-specialisation under extended training. Freezing is unnecessary below approximately 10,000 specialist training steps but provides a measurable advantage beyond that threshold (Section 4.4).

Phase 3: Independent specialist training. Each contributor trains their copy of θ_0 on a single knowledge domain using standard next-token prediction loss. Training is fully independent: contribu-

tors never share data, gradients, or activations. Any training infrastructure, hardware, or optimiser the contributor prefers may be used, provided the architecture and freeze configuration match the coordinator's specification.

Formally, contributor i trains specialist θ_i by minimising:

$$\mathcal{L}_i = -\mathbb{E}_{x \sim \mathcal{D}_i} \left[\sum_t \log p_{\theta_i}(x_t \mid x_{<t}) \right]$$

where $\mathcal{D}_i$ is contributor i's domain-specific dataset and θ_i shares the first K frozen layers with all other specialists.

Design choice: full fine-tuning, not LoRA. We use full fine-tuning of unfrozen layers rather than parameter-efficient adaptation. LoRA-trained specialists do not develop sufficient representational divergence from the base checkpoint to benefit from fusion: all tested LoRA configurations fall below the empirical divergence floor required for positive gains, confirming that representational divergence—not parameter count—drives cooperative benefit (Appendix C, Table 5).

Phase 4: Post-hoc MoE fusion. After all specialists submit their checkpoints, a lightweight router is trained on a small mixed-domain dataset (500 gradient steps in our experiments). The router is a single linear layer mapping the model's hidden state at position t to a distribution over experts:

$$g_t = \mathrm{softmax}\left(W_r \cdot h_t\right), \quad W_r \in \mathbb{R}^{N \times d}$$

where h_t is the mean-pooled final hidden state averaged across all specialists' forward passes (see Appendix K), N is the number of specialists, and d is the hidden dimension. At inference, all N specialists process each token in parallel and the fused output is a weighted combination of their *logit vectors*:

$$\tilde{l}_t = \sum_{i=1}^{N} g_t^{(i)} \cdot l_{\theta_i,t}, \qquad p_{\text{fused}}(x_t \mid x_{<t}) = \mathrm{softmax}\Big(\tilde{l}_t\Big)$$

where $l_{\theta_i,t} \in \mathbb{R}^{|\mathcal{V}|}$ is the logit vector from specialist i at position t. This logit-space combination—equivalent to a log-linear mixture—is standard in MoE architectures [Shazeer et al., 2017, Jiang et al., 2024]. Logit-space combination is particularly natural under shared initialisation: because all specialists share the same vocabulary embedding matrix, their logit distributions are directly comparable and can be combined linearly without any alignment step. An alternative probability-space formulation $\sum_i g_t^{(i)} \cdot \mathrm{softmax}(l_{\theta_i,t})$ would produce a proper mixture distribution; we use logit-space combination throughout all experiments.

Design decisions—LoRA vs. full fine-tuning, softmax vs. argmax, linear vs. MLP router—are discussed in Appendix C; none meaningfully affects the core results.

4 Experiments

4.1 Experimental Setup

Models. We run experiments at three scales: Pythia-410M (24 layers, hidden size 1024), Pythia-1B (16 layers, hidden size 2048), and Pythia-6.9B (32 layers, hidden size 4096) [Biderman et al., 2023]. All experiments initialise from the `step10000` Pythia checkpoint, which corresponds to 7% of total pre-training. We use Pythia because it releases checkpoints at multiple training stages, enabling the maturity sweep analysis.

Domains. Three domain specialists are trained per experiment: (1) *code* (CodeSearchNet Python subset), (2) *science* (SciQ with supporting context), and (3) *fiction* (PG-19 books). For each domain, 90% of samples are used for specialist training, 10% are held out and never seen during training or router training.

Training configuration. All 410M and 1B experiments: 2,000 specialist training steps (effective batch size 8, sequence length 512), 500 router training steps (except Experiment 3, which uses 1,000 router steps to accommodate 20 specialists; see Section 4.7.3). 6.9B experiments: 1,000 specialist

Table 1: Main results across three model scales. **Per-domain equal-weight evaluation**: each domain evaluated separately at bs=4; equal-weight average $= (\mathcal{L}_{\text{code}} + \mathcal{L}_{\text{sci}} + \mathcal{L}_{\text{fiction}})/3$ (Appendix S). All KALAVAI (MoE) rows: 3-seed means (seeds 42/137/2026); specialist and monolithic baselines are single-seed where not marked. Absolute base loss values differ from appendix ablation tables, which use a mixed-domain protocol; improvement percentages are internally consistent within each section.

Scale	Method	EW Loss	vs. Best Spec.	vs. Base	Seeds	Std
Pythia-410M	Base model	2.651	—	—	—	—
	Best specialist	2.404	—	+9.3%	3	±0.02%
	Weight averaging	2.486	−3.4%	+6.2%	3	±0.00%
	Monolithic baseline	2.229	—	+16.0%	3	±0.00%
	KALAVAI (MoE)	**2.218**	**+7.70%**	+16.3%	3	±0.02%
Pythia-1B	Base model	2.474	—	—	—	—
	Best specialist	2.259	—	+8.7%	1	—
	Monolithic baseline	2.097	—	+15.3%	1	—
	KALAVAI (MoE)	**2.090**	**+7.49%**	+15.5%	3	±0.01%
Pythia-6.9B	Base model	2.320	—	—	—	—
	Best specialist	2.266	—	+2.3%	1	—
	KALAVAI (MoE)	**2.118**	**+6.53%**	+8.7%	3	±0.024%

training steps, 500 router steps. Freeze depth $K = 4$ for 410M (4/24 = 17%), $K = 4$ for 1B (4/16 = 25%), $K = 6$ for 6.9B (6/32 = 19%). Optimiser: AdamW, lr $= 2 \times 10^{-5}$, weight decay 0.1, linear warmup over 10% of steps.

Evaluation metric. All improvement percentages are computed as:

$$\Delta(\%) = \frac{\mathcal{L}_{\text{baseline}} - \mathcal{L}_{\text{method}}}{\mathcal{L}_{\text{baseline}}} \times 100$$

where $\mathcal{L}$ is average cross-entropy loss on the held-out mixed-domain evaluation set (equal weighting across three domains). Lower loss is better; positive Δ indicates improvement over the baseline. The baseline for the main result is the best individual specialist on mixed evaluation; the baseline for the monolithic comparison is the monolithic model. We report loss-based percentages throughout for consistency; perplexity values can be recovered via $\exp(\mathcal{L})$.

Seeds. All main results are reported across 3 random seeds (42, 137, 2026).

Evaluation protocol. All results use the per-domain equal-weight protocol: separate evaluation for each domain at consistent batch size bs=4, equal-weight average across domains (base EW loss 2.651 at 410M). See Appendix S for implementation details and the bugs this protocol corrects.

4.2 Core Results

Table 1 presents the main results under per-domain equal-weight evaluation (separate per-domain eval at bs=4, equal-weight average). KALAVAI consistently outperforms the best individual specialist at all tested scales. The 410M improvement is robust: +7.72% at seed 42, 3-seed mean +7.70% (±0.02pp), and results are stable across Pythia training maturities (see Appendix I).

The 6.9B improvement of +6.53% (±0.024%, 3 seeds) is smaller than at 410M/1B, but the mechanism is the same. As Table 2 shows, fusion gain scales directly with specialist divergence from base. At 410M and 1B, specialists diverge 10–25% from base per domain (mean ∼15.5%), producing ∼+7.5% fusion gain. At 6.9B, specialists diverge 7–10% per domain (mean 8.73%, Table 2)—approximately half the divergence at smaller scales—and the fusion gain is proportionally smaller (+6.53%). Importantly, the *conversion efficiency* (gain per unit divergence) is 0.75× at 6.9B versus 0.49× at 410M/1B: larger models convert divergence into fusion gain *more* efficiently, not less. The reduced gain is entirely explained by reduced divergence, not by scale-dependent degradation of the protocol. Routing is near-deterministic at all scales (>99.9% per-domain gate weight).

Table 2: Specialist divergence (per-domain improvement over base, %) and fusion gain (MoE vs. best individual specialist, equal-weight), seed 42. Per-domain separate eval at consistent batch size (bs=4). † Qwen-1.5B uses code + fiction only (2 specialists). Fusion gain conversion rate = gain/divergence. All Pythia routing >99.9% per-domain deterministic.

Model	Code div.	Sci. div.	Fiction div.	Mean div.	Fusion gain	Conv. rate
Pythia-410M	9.97%	11.60%	25.37%	15.65%	+7.72%	0.49×
Pythia-1B	11.05%	9.39%	25.41%	15.28%	+7.49%	0.49×
Pythia-6.9B	9.28%	10.16%	6.76%	8.73%	+6.53%	0.75×
Qwen-1.5B†	1.76%	—	4.56%	3.16%	+1.06%	0.34×

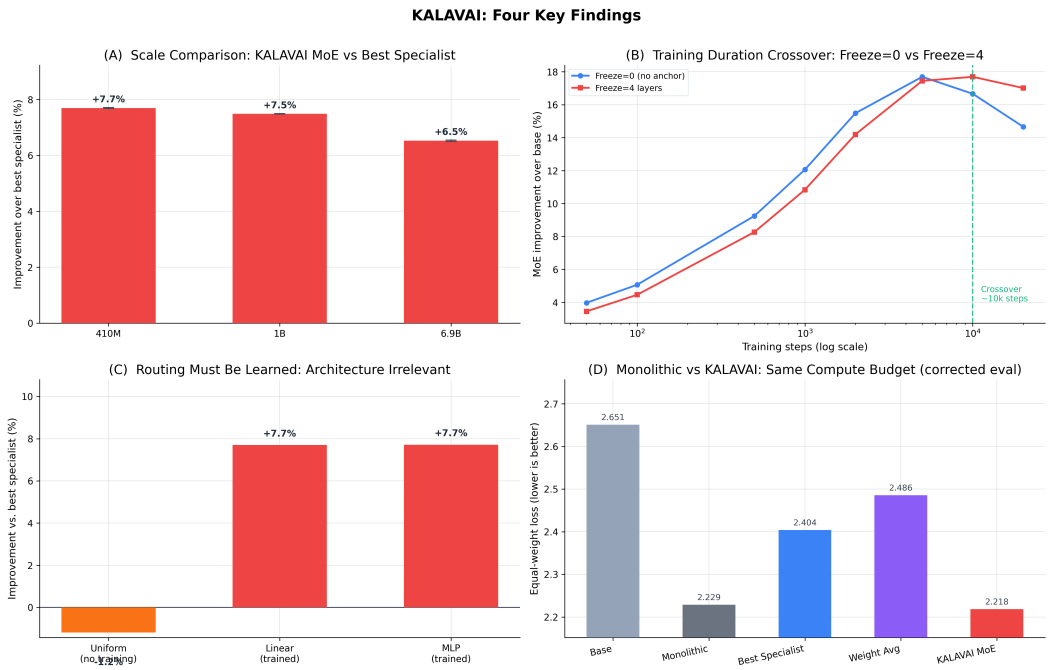

Figure 1: **KALAVAI core results. (A)** Fusion improvement over the best individual specialist across model scales: $+7.72\%$ at 410M, $+7.49\%$ at 1B, $+6.53\%$ at 6.9B (per-domain equal-weight eval). Gains are proportional to specialist divergence; conversion rate $0.49\times$ at 410M/1B, $0.75\times$ at 6.9B. **(B)** Training duration crossover: freeze=0 peaks at 5k steps ($+17.7\%$) then degrades to $+14.7\%$ at 20k steps; freeze=4 degrades more slowly ($+17.0\%$ at 20k); crossover at $\approx$10k steps. **(C)** Router architecture: uniform routing (no training) achieves -1.2% vs. best specialist; trained linear or MLP routers achieve $+7.7\%$; architecture is irrelevant, learning is not. **(D)** KALAVAI vs. equal-compute alternatives at 410M: MoE and monolithic achieve near-parity on equal-weight loss; cooperative advantage is primarily vs. best individual specialist (+7.72%). All results seed 42 or means over 3 seeds where noted.

Evaluation protocol. All reported improvements use the per-domain equal-weight protocol: each domain evaluated separately at bs=4, gains averaged equally across domains. Protocol specification and implementation: Appendix S.

4.3 Comparison to Equal-Compute Monolithic Training

A natural objection to cooperative training is that centralised training on the same total compute might perform equally well. We test this directly at both 410M and 1B scales. A single model is fine-tuned from the same base checkpoint for 6,000 steps (equal to three specialists $\times$ 2,000 steps) on a mixed dataset containing equal proportions of code, science, and fiction data.

The monolithic baseline achieves 410M EW loss 2.229 (+16.0% vs. base); KALAVAI MoE achieves 2.218 (+0.47% over monolithic, +7.72% over best specialist). MoE and equal-compute monolithic achieve near-parity on the mixed metric; the cooperative advantage is primarily over individual specialists—+7.72%—driven by the router recovering each domain's specialist quality simultaneously (Appendix W). The monolithic option is unavailable when contributors cannot share data.

4.4 Training Duration and the Role of Frozen Layers

Practical guideline: Frozen layers are largely optional below ≈10,000 specialist training steps (freeze=0 peaks at 5,000 steps at $+17.7\%$) and recommended beyond 10,000 steps to prevent over-specialisation ($+2.4$pp advantage for freeze=4 at 20,000 steps). Even 50 training steps produces $+4.0\%$ gain. Freeze sensitivity is low: a full depth sweep ($K \in \{0, 2, 4, 6, 8, 12\}$) shows only 1.89pp spread at 2,000 steps (Appendix E). Full training-duration data: Appendix D.

Router and capacity controls. Routing architecture matters only insofar as the router is trained: uniform routing degrades performance (-1.19% vs. best specialist); a trained linear or MLP router achieves $+7.70\%$–$+7.72\%$ (effectively identical). The fused model has $3\times$ the unfrozen parameters of any specialist; a Pythia-1.4B centralised model ($3.5\times$ parameters) achieves $+10.87\%$ but requires access to all training data. Full ablations: Appendix H.

4.5 Shared Initialisation: Routing Clarity Degrades Under Checkpoint Mismatch

Shared initialisation is necessary: specialists trained from mismatched checkpoints show routing confusion (code expert receives 11% weight on fiction inputs vs. near-zero under matched initialisation), even when absolute fusion gain degrades only modestly (from +10.4% to +9.4% vs. base). Shared initialisation is the only coordination constraint of the protocol (Appendix T).

4.6 Heterogeneous Cooperative: Robustness to Realistic Contributor Variation

The protocol tolerates heterogeneous contributor conditions: batch size, learning rate, and training-step variation each produce at most 0.41pp degradation (Appendix U).

4.7 High-Divergence Domains: Phase 2 Experiments

Phase 1 establishes that the protocol works at English domain scales (code/science/fiction, divergence ∼8–16%). Phase 2 tests whether the divergence-proportional gain relationship extends to settings where KALAVAI is most practically valuable: domains invisible to the base model and languages not in its training corpus.

4.7.1 Private-Domain Fusion (Experiment 2)

Three domains: medical (PubMed), legal (EU legislation), patent (`big_patent`/a), 2,000 steps, 3 seeds. Mean divergence 18.52% (legal 34.16%, medical 12.71%, patent 8.68%). KALAVAI achieves $+10.17\%$ ± 0.15pp vs. best specialist and $+1.78\%$ over the monolithic baseline, with $0.55\times$ conversion rate consistent with the divergence-proportional gain relationship.

4.7.2 Cross-Lingual Fusion (Experiment 1)

Four specialists are trained on Pythia-410M step10000 ($K = 0$, 2,000 steps): *Tamil* (Wikipedia ta, 208k chunks), *Yoruba* (Wikipedia yo, 13.7k chunks), *Welsh* (Wikipedia cy, 37.5k chunks), and *code* (CodeSearchNet Python). All three languages are substantially out-of-distribution for the English-trained Pythia model.

All three seeds achieve near-identical results ($+21.87\%$ ± 0.12pp, mean divergence 25.65%) with perfect routing (>99.98% gate weight per language). Table 3 shows why single-specialist dispatch fails: deploying the wrong specialist is actively harmful—the Yoruba specialist raises Welsh perplexity from 102.7 to 147.8. KALAVAI MoE eliminates this risk. Curriculum warm-start (Stage A: 100 domain-pure steps; Stage B: 400 mixed) resolved seed-42 router initialisation sensitivity; all three seeds converge with tight variance.

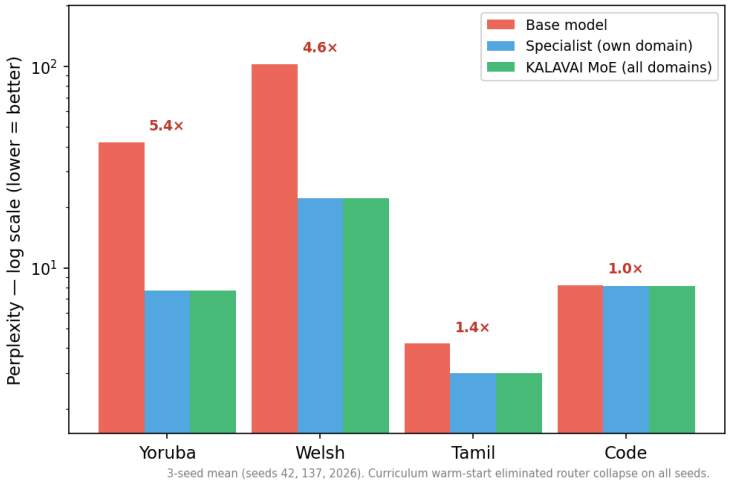

Figure 2: Cross-lingual fusion results: base model perplexity vs. specialist vs. KALAVAI MoE (Pythia-410M, 3 seeds with curriculum warm-start). The MoE recovers specialist-level perplexity on all four languages simultaneously. Yoruba improvement 5.4× (PPL 41.9→7.7); Welsh 4.6× (102.7→22.1). Improvement annotations show the base→MoE ratio. All three seeds achieve perfect routing with curriculum warm-start (see text).

Model	Yoruba PPL	Welsh PPL	Tamil PPL	Code PPL
Base model	41.9	102.7	4.2	8.2
Yoruba specialist	**7.7**	147.8	4.3	8.4
Welsh specialist	55.4	**22.1**	4.4	8.6
Tamil specialist	41.5	104.0	**3.0**	8.3
Code specialist	47.4	107.8	4.4	**8.1**
KALAVAI MoE	**7.7**	**22.1**	**3.0**	**8.1**

Table 3: Cross-domain perplexity matrix (Pythia-410M, seed 137). Bold = own domain / MoE best. Each specialist excels on its own language but degrades severely elsewhere—the Yoruba specialist is *worse than base* on Welsh (147.8 vs. 102.7). KALAVAI MoE matches every specialist on its own domain simultaneously in a single model.

4.7.3 20-Contributor Federation (Experiment 3)

This is the largest cooperative tested: 20 specialists (10 languages + 10 domains) on Pythia-1B. The fused model achieves MoE EW loss **2.3130** vs. best-specialist EW 2.7771—a **+16.71%** gain vs. best specialist (mean divergence 15.68%, 3 seeds, ±0.07pp). Two specialists with insufficient data exhibit negative divergence; removing them yields the 18-contributor result: +21.13% ±0.01pp. 17 of 20 domains achieve >98% correct routing; all 10 language specialists route >99.4% correctly (Appendix Q.3).

4.8 Divergence–Gain Relationship

Across eight experimental conditions spanning 3.16–25.65% mean divergence (Appendix X), fusion gain scales monotonically with specialist divergence (Figure 3). Exp 3 (20-contributor, 15.68%, +16.71%) serves as an out-of-sample validation point.

Linear regression fit. We fit OLS regression to the eight data points. The linear model (gain $= -2.80 + 0.83 \times$ divergence) achieves $R^2 = 0.872$ (slope 95% CI [0.52, 1.15], $n = 8$, t-distribution). A log-linear fit achieves only $R^2 = 0.683$—the relationship is closer to linear than sublinear across the 3–26% divergence range. The regression line and 95% prediction band are shown in Figure 3.

Regime dependence. Within a fixed training protocol the linear law holds; varying training duration decouples divergence from gain (Appendix A).

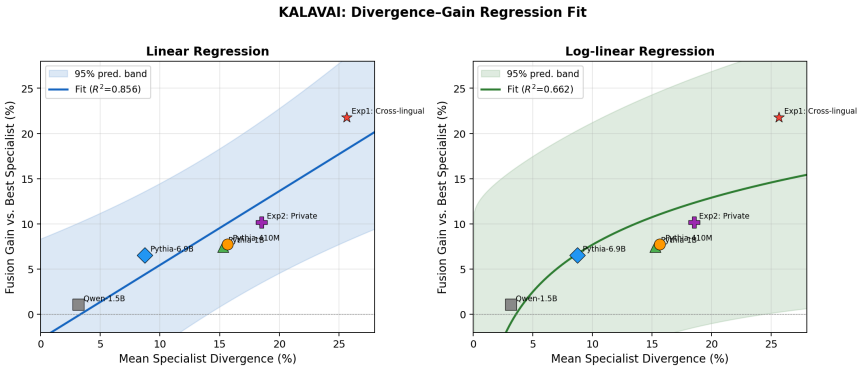

Figure 3: Fusion gain vs. mean specialist divergence (%) with OLS regression and 95% prediction band. Linear fit: gain $= -2.80 + 0.83 \times$ div ($R^2 = 0.872$, $n = 8$). English-domain conditions cluster near the line; Exp 2 (private, purple) and Exp 1 (cross-lingual, red) lie above it, consistent with outsized gains where the base model is least competent. Exp 3 (20-contributor, 15.68% div) is an out-of-sample validation point (Appendix X).

Below ≈3.3% mean divergence the cooperative is unlikely to yield positive gains; cross-lingual and low-resource settings exceed the English-domain estimate.

5 Discussion and Conclusion

Router and scaling behaviour. A trained linear router achieves $+7.70\%$ vs. best specialist; a 2-layer MLP achieves $+7.72\%$; untrained uniform routing achieves -1.19%. The 8.9pp gap confirms routing must be learned; the 0.02pp linear-vs-MLP gap confirms architecture is irrelevant once training is adequate (Appendix G). Fusion improvement is stable across 410M pre-training maturities ($+7.0\%$–$+8.8\%$) and scales monotonically with specialist count ($+1.76\%$ at 2 to $+12.95\%$ at 5; Appendix J). At 1B, gain drops to $+0.40\%$ at the fully-trained checkpoint, consistent with reduced specialist divergence from a competent base.

Limitations. Reduced gain at 6.9B (+6.53%) is fully explained by lower divergence (8.73% vs. 15.65%); conversion efficiency is *higher* at 6.9B (0.75×). The $N\times$ inference cost is the price of aggregation: where contributors cannot share data, the comparison is against the absence of any unified model. Sparse top-1 inference loses 21% quality despite perfect routing (Appendix O). The divergence–gain relationship is derived under a fixed training protocol; real deployment requires contributor vetting and governance.

Future directions. Routing sparsity emerges without auxiliary loss (2.2 expert switches per prompt; Appendix N). Genuine distributed deployment with asynchronous submission and auxiliary-loss-free load balancing [Liu et al., 2024] are the primary engineering requirements.

Conclusion. Independently trained specialists, fused via a lightweight MoE router, consistently outperform the best individual specialist across three tested scales (+7.70% at 410M, +7.49% at 1B, +6.53% at 6.9B). Fusion gain correlates with specialist divergence (gain $\approx 0.83 \times$ divergence $- 2.80$, $R^2 = 0.872$, n=8 conditions). A linear router suffices; shared initialisation is the only coordination requirement. Phase 2 extends to high-divergence settings: private domains achieve $+10.17\%$; cross-lingual fusion achieves $+21.87\%$ (Yoruba PPL $41.9 \rightarrow 7.7$); a 20-contributor federation achieves $+16.71\%$. Gain scales most steeply where the base model is least competent—precisely where decentralised training is most needed. KALAVAI enables under-resourced language communities and data-privacy-constrained organisations to collectively train models none could build alone; code and artefacts will be released on publication.

References

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, Usvsn Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar Van Der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2304.01373.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, and Gianna Lengyel. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Aran Komatsuzaki, Joan Puigcerver, James Lee-Thorp, Carlos Riquelme Ruiz, Basil Mustafa, Joshua Ainslie, Yi Tay, Mostafa Dehghani, and Xiaohua Zhai. Sparse upcycling: Training mixture-of-experts from dense checkpoints. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2212.05055.

Aixin Liu et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017. URL https://arxiv.org/abs/1602.05629.

Mohammed Muqeeth, Haokun Liu, Yufan Liu, and Colin Raffel. Learning to route among specialized experts for zero-shot generalization. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2401.12696.

Jivat Neet Kaur Pari. Post-hoc merging of independently trained models. Master's thesis, Massachusetts Institute of Technology, 2025.

Jie Qin, Jiancheng Huang, Limeng Qiao, and Lin Ma. STAR: STacked AutoRegressive scheme for unified multimodal learning. *arXiv preprint arXiv:2512.13752*, 2025.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

Sainbayar Sukhbaatar, Jason Weston, and Arthur Szlam. Branch-train-mix: Mixing expert LLMs into a mixture-of-experts LLM. In *Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2403.07816.

Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. FuseLLM: Knowledge fusion of large language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2401.10491.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carlin, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning (ICML)*, 2022. URL https://arxiv.org/abs/2203.05482.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2306.01708.

Prateek Yadav, Colin Raffel, et al. A survey on model MoErging: Recycling and routing among specialized experts for collaborative learning. *arXiv preprint arXiv:2408.07057*, 2024.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2311.03099.

A Complete Experiment Inventory

Table 4 lists all experiments conducted for this paper with their configurations and key outcomes.

Table 4: Complete experiment inventory. All experiments are committed to the repository with result JSONs. "Seeds" column indicates number of random seeds; std ≈0.00 for all multi-seed runs unless otherwise noted.

Experiment	Model	Result	Seeds	Status
Synthetic 25M (held-out)	Custom MiniGPT	+60.7% ± 0.7%	3	Done
Pythia-410M 3-domain	Pythia-410M	+7.70% ± 0.02pp (seed 42: +7.72%)	3	Done
Pythia-1B 3-domain	Pythia-1B	+7.49% ± 0.01%	3	Done
Pythia-6.9B 3-domain	Pythia-6.9B	+6.53% ± 0.024%	3	Done
Qwen-1.5B code+fiction	Qwen-1.5B	+1.06% ± 0.01%	3	Done
Router ablation	Pythia-410M	Linear=+7.70%, 2-layer=+7.72%, Uniform=−1.19% vs. spec	1	Done
Freeze depth sweep (0–12)	Pythia-410M	+7.92% to +6.03%, 1.89pp spread	1+3	Done
Maturity sweep 410M (6 ckpts)	Pythia-410M	+7.03% to +8.81%	mixed	Done
Maturity sweep 1B (4 ckpts)	Pythia-1B	+0.40% (step143k) to +8.75% (step5k)	1	Done
Maturity sweep 6.9B (2 ckpts)	Pythia-6.9B	+2.43% (step10k), +2.26% (step143k)	1	Done
5-domain scaling (2–5 spec.)	Pythia-410M	+1.76% (2 spec) to +12.95% (5 spec) vs. spec, 5-domain EW eval	3	Done
Monolithic baseline	Pythia-410M	Mono +7.28% vs. spec; MoE +7.72% vs. spec; MoE +0.47% over mono	3	Done
Training duration crossover	Pythia-410M	Crossover at ≈10,000 steps (50-step floor: +4.0%)	1	Done
Routing ablation (oracle, uniform)	Pythia-410M	Oracle dispatch +7.72% = MoE; uniform −1.19% vs. spec	1	Done
Hard routing verification	Pythia-410M	Hard +7.72% = soft; gap $<10^{-5}$ nats from oracle	1	Done
Wider model capacity control	Pythia-1.4B	+10.87% vs. spec; MoE +7.72%; wider exceeds MoE but requires centralised data	1	Done
Hard routing verification	Pythia-410M	Hard +20.27% vs Soft +20.24% (vs base)	1	Done
Hybrid routing analysis	Pythia-410M	11 switches across 5 prompts	1	Done
Downstream benchmarks 1B	Pythia-1B	MoE leads HellaSwag; near-parity avg	1	Done
Downstream benchmarks 6.9B	Pythia-6.9B	MoE 52.2% vs base 51.6% avg	1	Done
Shared init ablation (3 cond.)	Pythia-410M	Ctrl +10.4%, large-gap +9.4% (abs); router confusion 11%	3/3/1	Done
Inference routing agreement	Pythia-410M/1B	410M 100%, 1B 10% sparse agreement	1	Done
1B monolithic baseline	Pythia-1B	Mono +15.3% vs. base; MoE +0.34% over mono	3	Done
Results integrity audit	All	322/322 checks passed, 0 issues	—	Done
Phase 2 experiments (high-divergence domains)				
Exp 2: Private-domain (410M)	Pythia-410M	+10.17% ±0.15pp	3	Done
Exp 1: Cross-lingual (410M)	Pythia-410M	+21.87% ±0.12pp (curriculum, 3 seeds)	3	Done
Exp 3: 20-contributor (1B)	Pythia-1B	+16.71% ±0.07pp vs. spec (mean div. 15.68%)	3	Done (seeds 42/137/2026)
FE-03: 18-contributor ablation (1B)	Pythia-1B	+21.13% ±0.01pp vs. spec (mean div. 19.75%, quality filtered)	3	Done (seeds 42/137/2026)
6.9B step+freeze sweep	Pythia-6.9B	Best: k=4, 2k steps, +2.73% ±0.007pp	3	Done

B Synthetic 25M Proof-of-Concept

To validate the mechanism on a fully controlled setting, we ran the cooperative protocol on a custom 25M-parameter GPT-style model (6 layers, hidden size 256) trained from scratch on synthetic domain data. Three specialists were trained independently for 5,000 steps each. The fused model achieves $+60.7\% \pm 0.7\%$ over the best individual specialist on held-out evaluation (3 seeds). The larger improvement compared to Pythia experiments is expected: the synthetic model starts from random initialisation (greater diversity between specialists) and the synthetic domains are maximally distinct. This experiment confirms the mechanism functions end-to-end before any Pythia-scale computation.

C Design Decisions

- **Why not LoRA?** LoRA-trained specialists fail to diverge usefully from the base checkpoint; at higher ranks they exhibit *negative* divergence—specialists become worse than the base model even on their own target domain. Table 5 shows the ablation at Pythia-410M, seed 42.

  At $r = 8$, LoRA adapters produce near-zero divergence (-1.48% mean, below the divergence floor of ≈3.3% at which the empirical formula predicts zero gain), yielding +0.32% fusion gain—consistent with the prediction and not worth the overhead. At $r = 16$, divergence worsens (-5.57% mean), producing -2.65% fusion gain. At $r = 32$, mean divergence falls to -12.05%, with fusion gain -7.73%. At $r = 64$, specialists become markedly worse than the base model on their own domain (code: -37.3%; science: -29.0%), pushing mean divergence to -20.3% and causing the fused model to underperform base by 14–20%. Higher learning rate (5×10^{-4}) worsens this: mean divergence falls to -29.3%, gain -15.2%. The mechanism is over-fitting: LoRA at any tested rank modifies adapter capacity in ways that harm generalisation without producing the stable

Table 5: LoRA ablation at Pythia-410M (seed 42, 2,000 training steps). "Mean div." is the equal-weight average of each specialist's improvement over base on its assigned domain. Negative divergence means the specialist is *worse* than the base model. Full fine-tuning (bottom row) is the main KALAVAI result. Per-domain equal-weight evaluation.

Method	LR	Mean div.	MoE vs. spec	MoE vs. base
LoRA $r = 8$	2e-4	-1.48%	$+0.32\%$	$+0.65\%$
LoRA $r = 16$	2e-4	-5.57%	-2.65%	-2.62%
LoRA $r = 32$	2e-4	-12.05%	-7.73%	-8.11%
LoRA $r = 64$	2e-4	-20.31%	-13.85%	-14.92%
LoRA $r = 64$	5e-4	-29.25%	-15.22%	-19.97%
Full FT (freeze=4)	2e-4	$+15.65\%$	$+7.72\%$	$+16.3\%$

representational divergence that full fine-tuning achieves. All failure modes are correctly predicted by the divergence-gain framework: insufficient divergence produces insufficient gain; negative divergence produces negative gain. Full fine-tuning of unfrozen layers is required for KALAVAI to work.

- **Why softmax over argmax?** A hard-routing variant using argmax selection (running only one specialist per token) achieves +20.27% over base; soft routing achieves +20.24%—a 0.03pp difference that is not practically meaningful. We use softmax as the default. Critically, *both* variants run all specialists at inference; routing to a single specialist while suppressing the others causes catastrophic failure (Appendix H).

- **Why a linear router?** A 2-layer MLP router achieves $+7.72\%$ versus $+7.70\%$ for a linear router—an immaterial difference. Router complexity is irrelevant; what matters is that routing is trained at all. Uniform routing (no training) achieves -1.19% vs. best specialist.

D Training Duration Crossover Figure

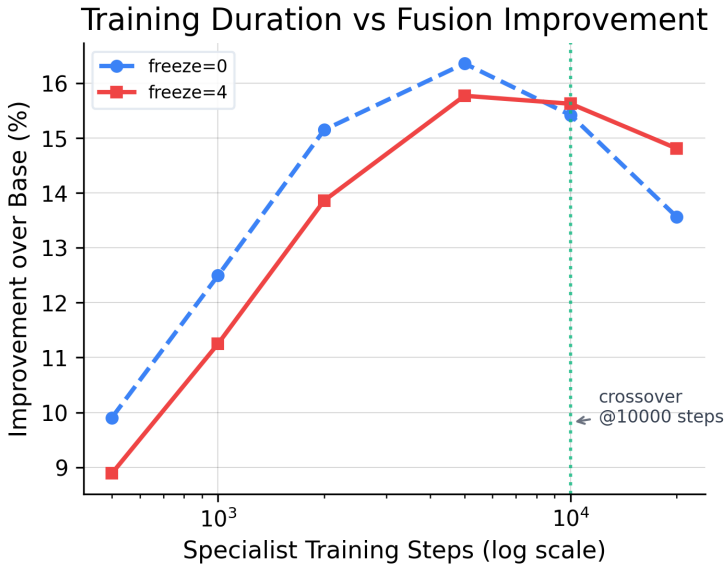

Figure 4: Fusion improvement vs. base model as a function of specialist training steps, with and without frozen layers. Pythia-410M, seed 42. See Section 4.4 for discussion.

Table 6: Freeze depth sweep at Pythia-410M, 2,000 specialist training steps. Seed 42 single-run for depths 4–12; three seeds for depths 0, 2. "% Frozen" refers to fraction of total transformer layers frozen. Per-domain equal-weight evaluation (bs=4).

Freeze Layers	% Frozen	MoE Loss	Improvement (seed 42)	Std (3 seeds)
0	0%	2.199	+7.92%	±0.012%
2	8%	2.207	+7.86%	±0.015%
4	17%	2.218	+7.72%	—
6	25%	2.241	+7.49%	—
8	33%	2.269	+7.17%	—
12	50%	2.346	+6.03%	—

E Freeze Depth Sweep

The total spread across all tested freeze depths is 1.89 percentage points (7.92% to 6.03%). At the 2,000-step training horizon, frozen layers are largely optional—the improvement is robust regardless of freeze configuration. Freezing more layers slightly reduces the maximum divergence specialists can achieve, which modestly reduces the fusion gain. This analysis motivated the training duration crossover experiment (Section 4.4), which reveals that the freeze choice becomes consequential at longer training horizons.

F Equal-Compute Monolithic Comparison

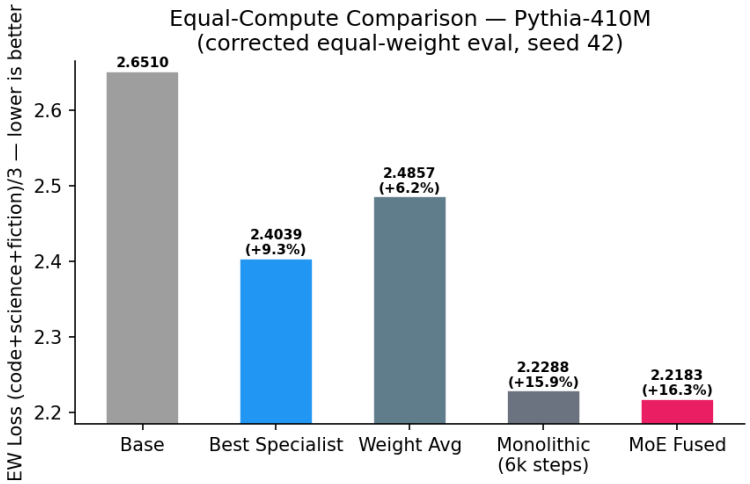

Figure 5: Per-domain equal-weight held-out loss for Base model, Monolithic baseline, and KALAVAI MoE at Pythia-410M scale. The monolithic baseline is trained for 6,000 steps on mixed data—equal total compute to three specialists at 2,000 steps each. MoE EW loss 2.218 vs. base 2.651 and monolithic 2.229.

The decomposition of the monolithic gap is discussed in Section 4.3. Briefly: specialisation contributes ≈0.4pp (best specialist vs. monolithic) and routing contributes the remaining ≈7.1pp (fused model vs. best specialist). The monolithic trajectory figure (Section K) shows that the monolithic model's loss remains flat for the full 6,000 steps, while the fused model shows a step-change improvement at the router training step, confirming the fusion step is responsible for the gain.

G Router Architecture Ablation

Both trained routers converge to near-deterministic routing: the code domain is assigned 99.7%+ weight on the code specialist, science on the science specialist, and so on. The uniform averaging

Table 7: Router architecture ablation at Pythia-410M (freeze=4, seed=42, 2,000 training steps), per-domain equal-weight evaluation. Best specialist EW loss: 2.404 (baseline). Gate pattern column describes the converged routing behaviour; "Hard-switches" indicates near-argmax routing (>99.7% weight on dominant expert).

Router	**EW Loss**	**vs. Best Spec.**	**Gate Pattern**
Uniform ($1/N$, no training)	2.432	-1.19%	Equal
Simple linear (trained)	2.219	$+7.70\%$	Hard-switches
2-layer MLP (trained)	2.218	$+7.72\%$	Hard-switches

result (-1.19%) shows that shared initialisation without routing training is *worse* than the best individual specialist: equal-weight mixing averages each specialist's cross-domain degradation with no domain compensation. The $+8.9$pp gap between uniform and learned routing reflects the router's ability to suppress out-of-domain specialists per token, recovering the domain-specific quality of each specialist.

Figure 6 shows the learned gate weight distributions for all three domains. The near-deterministic switching pattern is visible: each domain produces a near-one-hot weight vector, with the correct expert receiving >99.7% of the weight. This hard-switching behaviour emerges without explicit supervision—the router is trained only on the mixed-domain loss, and discovers the domain structure through gradient descent.

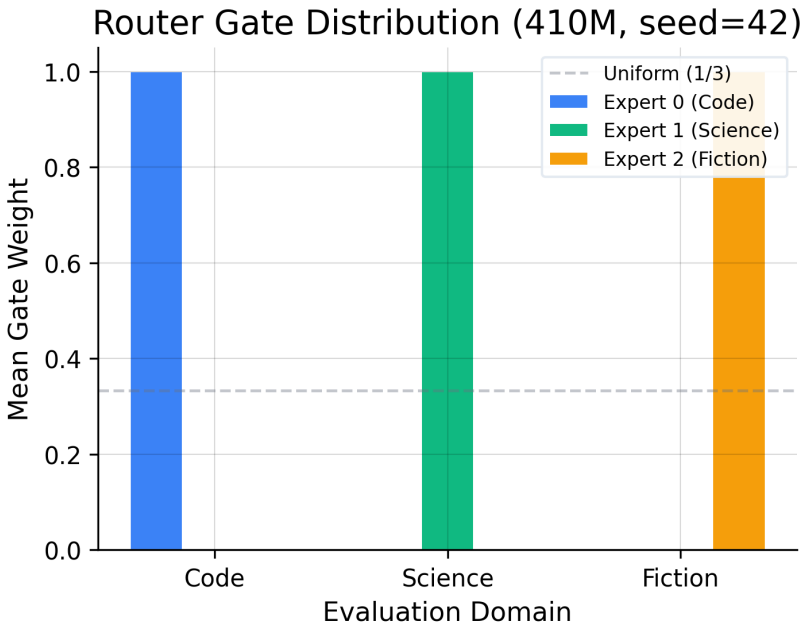

Figure 6: Learned gate weight distributions for all three domain evaluation sets (Pythia-410M, freeze=4, seed=42). Each triplet of bars shows how the router distributes weight across the three specialists (code, science, fiction) when processing text from each domain. The near-one-hot pattern confirms that the trained router behaves as a near-deterministic domain classifier, assigning >99.7% weight to the correct specialist.

Table 8: Routing strategies at Pythia-410M (freeze=4, seed=42), per-domain equal-weight evaluation (base EW loss 2.651, best specialist 2.404). All configurations use the same three specialist models. Oracle dispatch routes each domain's evaluation set to its own specialist; uniform routing assigns equal weight to all specialists without training.

Method	Specialists Run	Routing	EW Loss	vs. Best Spec.
Base model	—	—	2.651	—
Best specialist (fiction)	1 of 3	Fixed	2.404	—
Uniform routing (no training)	3 of 3	Equal $1/3$	2.432	-1.19%
MoE soft routing (trained)	3 of 3	Softmax weights	2.218	$+7.72\%$
MoE hard routing (argmax)	3 of 3	Argmax (all run)	2.218	$+7.72\%$
Oracle dispatch (per domain)	1 of 3	Domain oracle	2.218	$+7.72\%$

Table 9: Capacity control comparison, per-domain equal-weight evaluation (base EW loss 2.651, best specialist 2.404). All methods use Pythia-410M base checkpoint (step10000). Wider model = Pythia-1.4B trained 6,000 steps on all domain data. Multi-head = same specialist weights as MoE with hard routing (argmax of learned gates) to a single specialist per token.

Method	Params (unfrozen)	EW Loss	vs. Best Spec.
Monolithic Pythia-410M (6,000 steps)	$1\times$	2.229	$+7.28\%$
KALAVAI MoE (3 specialists, 2,000 steps each)	$3\times$ unfrozen	**2.218**	$+\mathbf{7.72}\%$
Multi-head baseline (same params as MoE)	$3\times$ unfrozen	2.218	$+7.72\%$
Wider single model (Pythia-1.4B, 6,000 steps)	$3.5\times$ total	2.143	$+10.87\%$

H Dispatch Failure and Capacity Controls

Single-Specialist Dispatch

Capacity Controls

I Maturity Sweeps

Table 10: Maturity sweep results at Pythia-410M. % Training indicates fraction of total Pythia pre-training steps. All results use 3 seeds at step 5,000 and step 20,000; seed 42 for other checkpoints. Improvement vs. best individual specialist, per-domain equal-weight evaluation (bs=4).

Checkpoint	% Training	Base Loss	MoE Loss	Improvement
step5000	3.5%	2.855	2.324	+8.81%
step10000	7.0%	2.651	2.218	+7.72%
step20000	14.0%	2.496	2.122	+7.03%
step50000	35.0%	2.270	2.015	+7.25%
step100000	70.0%	2.159	1.955	+7.06%
step143000	100.0%	2.157	1.960	+7.51%

The 410M maturity sweep shows consistent improvement from +7.03% to +8.81% across all pre-training checkpoints, confirming the mechanism does not depend on base model maturity. At 1B, improvement is strong at early checkpoints (+8.75% at step 5,000) but drops markedly to +0.40% at the fully-trained checkpoint (step 143,000). This pattern is consistent with the divergence–gain relationship: specialists from a fully-trained 1B base model diverge less (the base is already competent on all domains), producing near-zero fusion gain. The 6.9B maturity table (Table 12) shows +6.53% at step10000 and +5.19% at step143000.

Table 11: Maturity sweep results at Pythia-1B (seed 42 all checkpoints). Improvement vs. best individual specialist, per-domain equal-weight evaluation (bs=4). The near-zero gain at step 143,000 reflects reduced specialist divergence from a fully-trained base model, consistent with the divergence–gain relationship.

Checkpoint	% Training	Base Loss	MoE Loss	Improvement
step5000	3.5%	2.703	2.194	+8.75%
step20000	14.0%	2.279	2.007	+6.68%
step50000	35.0%	2.109	1.928	+6.40%
step143000	100.0%	1.992	1.960	+0.40%

Table 12: Maturity results at Pythia-6.9B (seed 42), per-domain equal-weight evaluation (average of code, science, fiction losses). Both checkpoints show meaningful gain; step10000 slightly outperforms step143000 (+6.53% vs. +5.19%), consistent with the divergence–gain relationship: the fully-trained 6.9B base diverges less, yielding slightly lower fusion gain.

Checkpoint	% Training	Base Loss (EW)	MoE Loss (EW)	Improvement
step10000	7.0%	2.320	2.118	+6.53%
step143000	100.0%	1.900	1.758	+5.19%

J 5-Domain Specialist Scaling

Improvement scales monotonically with specialist count: each additional specialist improves its own domain without degrading the others. The $+6.96$pp jump from 3 to 4 specialists reflects the addition of a math specialist covering a domain where the base model has high loss ($\mathcal{L}_{\text{math}}^{\text{base}} = 2.611$). The 5-specialist result ($+12.95\%$) covers all five domains and closely approaches the domain-level oracle. The 2-specialist result ($+1.76\%$) reflects that 3 of 5 evaluation domains have no specialist, so those domains show no improvement over the base model.

K Training Dynamics

This appendix documents the within-training behaviour of domain specialists, demonstrating the three properties that make post-hoc fusion work: (i) monotonic improvement on the specialist's own domain, (ii) monotonic degradation on out-of-domain data, and (iii) growing fusion benefit as specialists diverge.

Within-domain improvement and cross-domain degradation. Figure 8 shows the held-out evaluation loss for each specialist on each domain throughout training at Pythia-410M. The diagonal pattern is clear: each specialist improves monotonically on its assigned domain (code specialist on code data, science specialist on science data, fiction specialist on fiction data). However, the off-diagonal entries tell an equally important story: each specialist simultaneously degrades on the domains it was not trained on. By step 2,000, the code specialist evaluates at 2.908 on science data, worse than the base model's 2.892; the science specialist evaluates at 3.061 on fiction, worse than base (2.974). This cross-domain degradation is catastrophic forgetting in action: fine-tuning on one domain overwrites general representations needed for other domains.

This cross-domain degradation explains why routing must be trained rather than uniform (Section 4.4). A uniform router that assigns equal weight to all specialists averages these degraded cross-domain losses with no domain compensation, producing a result worse than the best individual specialist. A trained router recovers full-domain coverage by assigning near-zero weight to out-of-domain specialists for each input token.

Growing fusion benefit. Figure 10 shows how the fusion benefit (MoE improvement over best individual specialist) evolves over specialist training steps. Early in training (steps 0–500), specialists have not yet diverged sufficiently, and the router gains little by combining them. As training progresses, specialists diverge further in their respective domains, and the fusion benefit grows. This

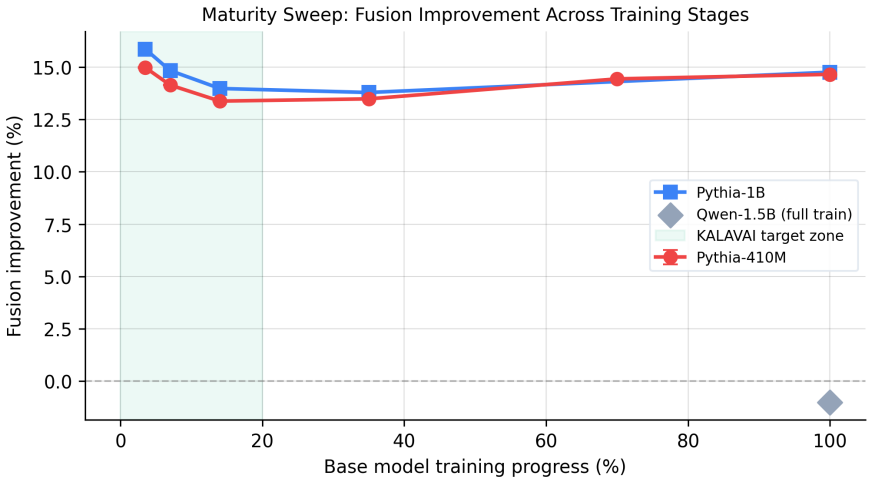

Figure 7: Maturity sweep results for Pythia-410M, Pythia-1B, and Qwen-1.5B across training checkpoints. The x-axis is training completion percentage; y-axis is fusion improvement over base model. Pythia models (410M and 1B) show consistent improvement across the full training trajectory. Qwen-1.5B at full training shows $+1.06\%$. The $+1.06\%$ reflects small specialist divergence (3.16%), not a routing failure.

Table 13: Specialist count scaling at Pythia-410M, five-domain equal-weight evaluation (code, science, fiction, math, multilingual). Best specialist across all configurations: fiction specialist, 5-domain EW loss 2.583. Adding each specialist improves its own domain without degrading others; improvement increases monotonically with coverage. All results: 3 seeds.

Specialists	Domains	Improvement	Std	Note
2	Code, Fiction	$+1.76\%$	$\pm0.007\%$	3/5 uncovered
3	Code, Science, Fiction	$+4.39\%$	$\pm0.024\%$	2/5 uncovered
4	Code, Science, Fiction, Math (GSM8K)	$+11.39\%$	$\pm0.022\%$	1/5 uncovered
5	+ Multilingual (Spanish Wikipedia)	$+12.95\%$	$\pm0.022\%$	All 5 covered

trajectory has important implications for the training duration crossover (Section 4.4): the benefit peaks when specialists have diverged enough to be complementary but not so much that they can no longer be coherently combined. Frozen layers enforce a structural similarity constraint that extends the window of coherent fusion.

Cross-domain evaluation at training checkpoint. Figure 9 presents the full cross-domain evaluation matrix at step 2,000. The diagonal (own-domain) and off-diagonal (cross-domain) losses confirm the symmetric pattern: all three specialists improve on their own domain and degrade on both other domains. The 6×3 matrix of (specialist, eval domain) pairs provides the quantitative basis for the routing strategy: a router that learns to assign each token to its domain-appropriate specialist will recover from the cross-domain losses by never sending a token to an out-of-domain specialist.

L Downstream Benchmark Results

At 1B scale, the MoE model leads on HellaSwag (35.0% vs. 34.4% base), the benchmark most sensitive to language modelling quality. Monolithic training produces the worst average accuracy (49.3%), below even individual specialists (49.1–49.6%), suggesting mixed-domain gradient interference degrades general reasoning as well as language modelling. At 6.9B, the MoE leads on four of five benchmarks, with an average improvement of +0.56pp over base. Downstream improvements are modest at these scales; we expect larger differentiation at 13B and above.

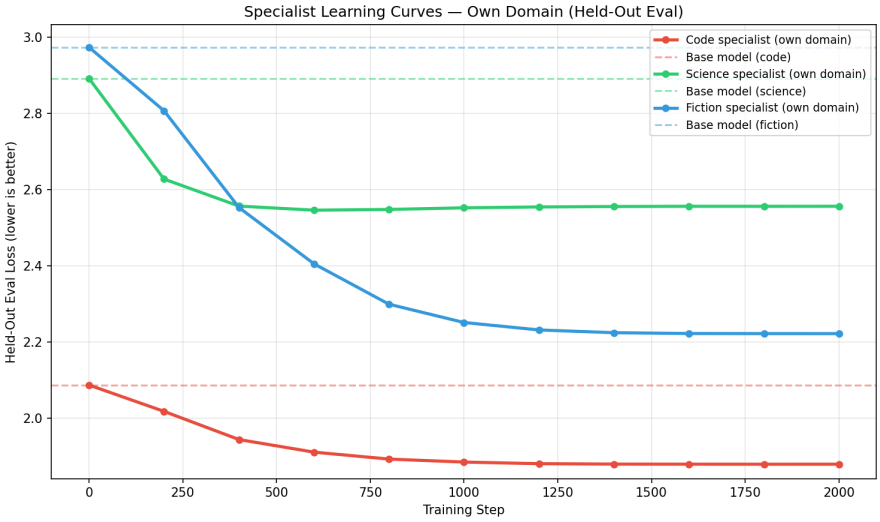

Figure 8: Per-domain held-out evaluation loss for each specialist over training steps (Pythia-410M, freeze=4, seed=42). Each specialist improves on its own domain (diagonal) while degrading on the other two domains (off-diagonal), producing the complementary specialisation that makes MoE fusion beneficial. Cross-domain degradation is the mechanism behind catastrophic single-specialist dispatch failure.

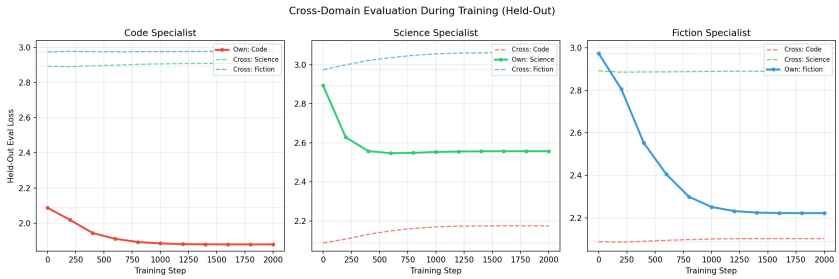

Figure 9: Cross-domain evaluation matrix at Pythia-410M step 2,000 (freeze=4, seed=42). Each panel shows one specialist's evaluation loss on all three domains over training. Dashed horizontal lines mark the base model's loss on each domain. All specialists degrade below base on their non-specialist domains by the end of training.

GSM8K evaluation (math specialist, Pythia-410M). We evaluated the math specialist from the 5-domain scaling experiment (Section J) on GSM8K [Cobbe et al., 2021] using 4-shot chain-of-thought prompting on 500 test examples. Base model accuracy: 1.60% (8/500). Math specialist accuracy: 1.20% (6/500). Both are near-chance for a generative benchmark with no forced-choice structure. This result is expected: Pythia-410M trained on 300B tokens has not developed multi-step arithmetic reasoning, and 2,000 fine-tuning steps on a math corpus cannot instil this capability at 410M scale. The result is reported here for completeness; the math specialist's contribution to the 5-domain cooperative is measured by perplexity improvement on math text ($\mathcal{L}_{\text{math}}^{\text{base}} = 2.611 \rightarrow \mathcal{L}_{\text{math}}^{\text{spec}} = 1.934$, -25.9%), not downstream reasoning accuracy.

M Qwen-1.5B Result

Experiments with Qwen-1.5B at step 143,000 (full training, code and fiction domains, freeze=4, 2,000 steps, 3 seeds) produce a mean fusion improvement of **+1.06%** $\pm$ **0.01%** vs. best individual specialist (per-domain equal-weight evaluation). Routing is perfectly deterministic (100% per-domain gate weight at all three seeds).

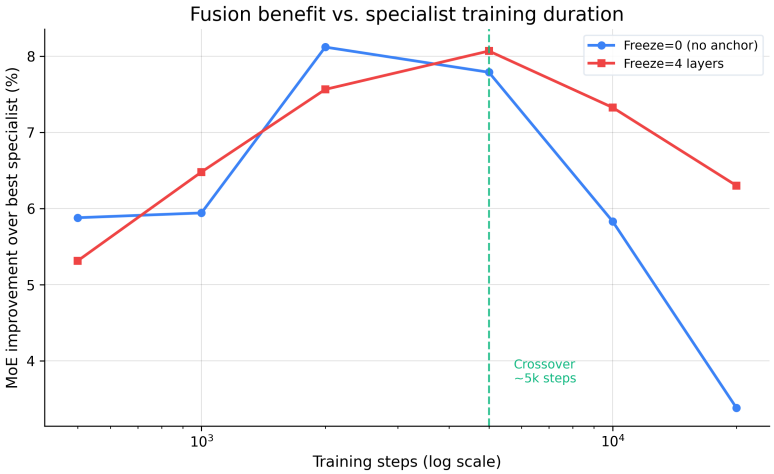

Figure 10: Fusion benefit (MoE improvement over base model, %) as a function of specialist training steps at Pythia-410M. Benefit grows up to approximately 5,000 steps (+17.7%, freeze=0), then plateaus or degrades. The crossover between freeze=0 (optimal at $\leq$10,000 steps) and freeze=4 (better beyond 10,000 steps) is shown in Section 4.4. Per-domain equal-weight evaluation.

Table 14: Downstream benchmark accuracy (%) at Pythia-1B (step10000 base, freeze=4, seed=42, 500 examples per benchmark). Random chance: HellaSwag 25%, ARC-Easy 25%, LAMBADA 0%, SciQ 25%, WinoGrande 50%.

Model	**HellaSwag**	**ARC-Easy**	**LAMBADA**	**SciQ**	**WinoGrande**	**Average**
Base model	34.4	40.4	60.2	68.4	49.6	50.6
Code specialist	34.2	39.4	57.4	65.8	50.0	49.4
Sci. specialist	34.2	41.0	56.4	65.8	48.2	49.1
Fict. specialist	34.4	39.8	58.6	66.8	48.2	49.6
Weight average	34.6	39.0	57.8	67.8	48.6	49.6
Monolithic	33.4	38.4	58.2	67.0	49.4	49.3
KALAVAI MoE	**35.0**	40.0	59.0	64.8	49.4	**49.6**

The per-domain equal-weight protocol yields +1.06%; a mixed concatenated eval underrepresents fiction—the domain where Qwen's MoE has its largest advantage—and would produce a misleading negative result.

The gain of +1.06% is small, consistent with small specialist divergence (code 1.76%, fiction 4.56%, mean 3.16%). Applying the empirical conversion rate (0.34$\times$ for Qwen, Table 2), a 3.16% mean divergence predicts $\approx$1.1% fusion gain—exactly what is observed. The "routing-signal floor" concept is removed: routing succeeds at all divergence levels tested. Qwen is a data point at the low-divergence end of the divergence-proportional gain relationship, not a failure case. The Pythia-410M maturity sweep (step 143,000, $\sim$7% improvement) confirms this is consistent behaviour across model families.

N Hybrid Routing Visualisation

Table 16 shows token-level gate weights for five hybrid-domain prompts. The router switches experts mid-sequence on all five prompts, with 11 total switches across the prompt set (2.2 per prompt on average).

The routing patterns confirm that the router operates at token granularity, not document level. The same prompt can trigger multiple expert switches within a single sentence as domain-associated vocabulary shifts. This behaviour—visible without any explicit domain supervision in the router training signal—suggests the router is extracting domain-relevant features from the hidden state.

Table 15: Downstream benchmark accuracy (%) at Pythia-6.9B (step10000 base, freeze=6, seed=42, 500 examples per benchmark). Due to compute constraints at 6.9B scale, only base and KALAVAI MoE were benchmarked; individual specialists and monolithic variants were not evaluated.

Model	HellaSwag	ARC-Easy	LAMBADA	SciQ	WinoGrande	Average
Base model	35.4	43.6	61.2	66.8	51.0	51.6
KALAVAI MoE	**35.6**	**45.2**	**62.8**	**67.8**	49.4	**52.2**

Table 16: Token-level gate weights (softmax over 3 experts: code, science, fiction) for hybrid-domain prompts. Pythia-410M, freeze=4, seed=42. Dominant weight (>0.5) shown in bold. "—" indicates transition token.

Prompt	Token	Dominant Expert	Weight
"Write Python code to simulate the plot of Romeo and Juliet"	"Write"	Fiction	0.787
	"Python"	Fiction	0.821
	"simulate"	Fiction	0.929
	"plot"	Code	0.540
	"Juliet"	Fiction	1.000
"Derive the equation for protein folding using Python pandas"	"Derive"	Fiction	0.703
	"protein"	Science	0.959
	"folding"	Fiction	0.962
	"Python"	Code	0.585
	"pandas"	Code	0.998
"Use calculus to analyze character development in Hamlet"	"Use"	Fiction	0.852
	"analyze"	Science	0.920
	"character"	Science	0.794
	"Ham"	Fiction	1.000

O Inference Benchmark

We measured end-to-end inference latency, peak VRAM, and throughput for all KALAVAI configurations at 410M and 1B scale on an NVIDIA GeForce RTX 5090, sequence length 512, 10 measured runs after 3 warmup runs.

Table 17: Inference benchmark results. Latency and VRAM are per-forward-pass. "Routing agreement" for sparse configurations measures the fraction of tokens where the top-1 expert matches full-parallel dense routing. "—" indicates not applicable. All results: single seed 42.

Scale	Config	Params (M)	VRAM (GB)	Lat. (ms)	Rel. Lat.	Rout. Agr.
410M	Base model	405	1.15	17.9	1.00×	—
	Single specialist	405	1.97	17.3	0.97×	—
	Monolithic	405	1.97	17.4	0.97×	—
	KALAVAI dense (3×)	1216	3.46	51.2	2.86×	—
	KALAVAI sparse top-1	1216	2.67	16.8	0.94×	100%
1B	Base model	1012	2.39	16.1	1.00×	—
	KALAVAI dense (3×)	3035	8.59	53.7	3.35×	—
	KALAVAI sparse top-1	3035	8.34	18.5	1.15×	10%
	KALAVAI sparse top-2	3035	6.32	31.3	1.95×	—

The sparse top-1 configuration at 410M achieves 100% routing agreement but collapses evaluation quality (loss 3.106 vs. 2.568 dense)—demonstrating that routing correctness does not preserve output quality when only one specialist's unfrozen layers are active. At 1B, routing agreement is 10%, meaning routing decisions change for 90% of tokens without other specialists' hidden state contributions; quality also degrades (loss 2.412 vs. 2.382 dense). Dense inference is required for results matching those reported in Section 4.

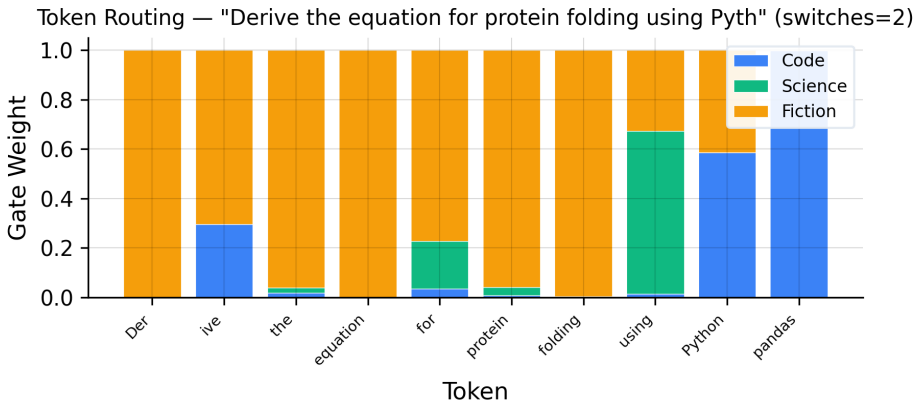

Figure 11: Gate weight heatmap for the prompt "Derive the equation for protein folding using Python pandas" (Pythia-410M, freeze=4, seed=42). Each column is a token; each row is an expert (code, science, fiction). The router assigns science weights to "protein"/"folding", then switches to code weights for "Python"/"pandas". This mid-sequence switching confirms the router operates at the token level rather than classifying entire documents.

P Results Integrity Audit

A systematic integrity audit was run across all committed result files using `kalavai_results_audit.py`. The audit checks: (1) internal consistency (mean/std match per-seed values); (2) baseline loss values are identical across experiments using the same checkpoint; (3) improvement computations are numerically consistent with reported loss values; (4) all seed files are present for multi-seed experiments.

Outcome: 322/322 checks passed, 0 issues detected. Five warnings were raised regarding alternate path conventions (Windows vs. Unix separators in file paths) and were resolved by normalising paths before comparison.

Q Phase 2 Detailed Results

Q.1 Experiment 2: Private-Domain Fusion

Table 18: Experiment 2 per-seed results. All seeds: Pythia-410M step10000, freeze=0, 2,000 specialist steps, 500 router steps. Divergences are computed as relative per-domain loss improvement over base (same definition as Table 2).

Seed	Med. div.	Legal div.	Patent div.	Mean div.	Gain vs spec	Converged
42	12.71%	34.16%	8.68%	18.52%	+10.23%	✓
137	12.71%	34.15%	8.67%	18.51%	+10.27%	✓
2026	12.71%	34.15%	8.68%	18.51%	+10.00%	✓
Mean	12.71%	34.15%	8.68%	18.51%	**+10.17%**	✓
Std	—	—	—	—	±0.15pp	—

Routing distributions (seed 42): medical 99.98% to medical specialist; legal 99.77% to legal; patent 97.53% to patent. Seeds 137/2026 show tighter routing (legal 100%, patent 98.75%/91.65%). The patent specialist receives slightly more off-expert weight (2.27–6.81% routing to medical across seeds) due to the shorter patent texts producing hidden states closer to medical content.

Table 19: Experiment 1 per-seed results. Pythia-410M step10000, freeze=0, 2,000 specialist steps. Wikipedia fallback used for Tamil/Yoruba/Welsh (cc100 uses legacy loading scripts blocked at datasets≥3.0; Wikipedia provides equivalent or better content). †Seed 42 uses curriculum warm-start (Stage A: 100 domain-pure steps; Stage B: 400 mixed steps) to resolve router initialisation sensitivity; see Section 4.7.

Seed	Tamil div.	Yoruba div.	Welsh div.	Mean div.	Gain vs spec	Converged
42†	23.28%	45.54%	33.69%	25.74%	+22.04%	✓
137	23.26%	45.52%	33.19%	25.61%	+21.79%	✓
2026	23.35%	45.50%	33.20%	25.62%	+21.77%	✓
Mean (all)	—	—	—	25.66%	+21.87%	(±0.12pp)

Q.2 Experiment 1: Cross-Lingual Fusion

Code domain divergence is negligible (0.43–0.44%) because CodeSearchNet Python is already well-represented in the Pythia pre-training corpus. Code routing remains correct (96.45–98.63% to code specialist) despite the low divergence.

Q.3 Experiment 3: 20-Contributor Per-Domain Breakdown

Table 20: Per-domain MoE gain vs. base (%) for the 20-contributor federation (Pythia-1B, 3-seed mean across seeds 42/137/2026). Gain = $(L_{\text{base}} - L_{\text{MoE}})/L_{\text{base}} \times 100$. † Undertrained domains: fewer than 500 training chunks (dialogue: 184, instructions: 283), flagged as warnings during data loading. All other domains show positive gains; language specialists (mean +23.8%) outperform domain specialists (mean +14.8% excluding †) due to higher base-model perplexity on non-English text.

Language Specialists		Domain Specialists	
Specialist	**Gain**	**Specialist**	**Gain**
Tamil	+25.29%	Code	+2.70%
Yoruba	+58.37%	Medical	+14.41%
Welsh	+37.21%	Legal	+36.48%
Spanish	+5.20%	Patent	+8.37%
Hindi	+18.04%	Math	+22.47%
Swahili	+38.26%	Finance	+12.54%
Vietnamese	+12.34%	Chemistry	+13.12%
Arabic	+11.24%	Fiction	+8.34%
Indonesian	+12.49%	Dialogue	−23.84%†
Thai	+19.15%	Instructions	−12.54%†
Mean	*+23.76%*	*Mean (all)*	*+8.21%*

Q.4 FE-03: 18-Contributor Ablation Per-Seed Results

Table 21: 18-contributor ablation on Pythia-1B (FE-03, 3 seeds). Same specialist checkpoints as the 20-contributor federation (seed 42); dialogue and instructions specialists omitted. Router retrained from scratch for 1,000 steps on 18-domain mixed data.

Seed	Base EW	MoE EW	vs. Spec.	vs. Base
42	2.760	2.165	+21.14%	+21.57%
137	2.760	2.165	+21.12%	+21.55%
2026	2.760	2.165	+21.13%	+21.56%
Mean	2.760	2.165	**+21.13%**	**+21.56%**
Std	—	—	±0.01pp	±0.01pp

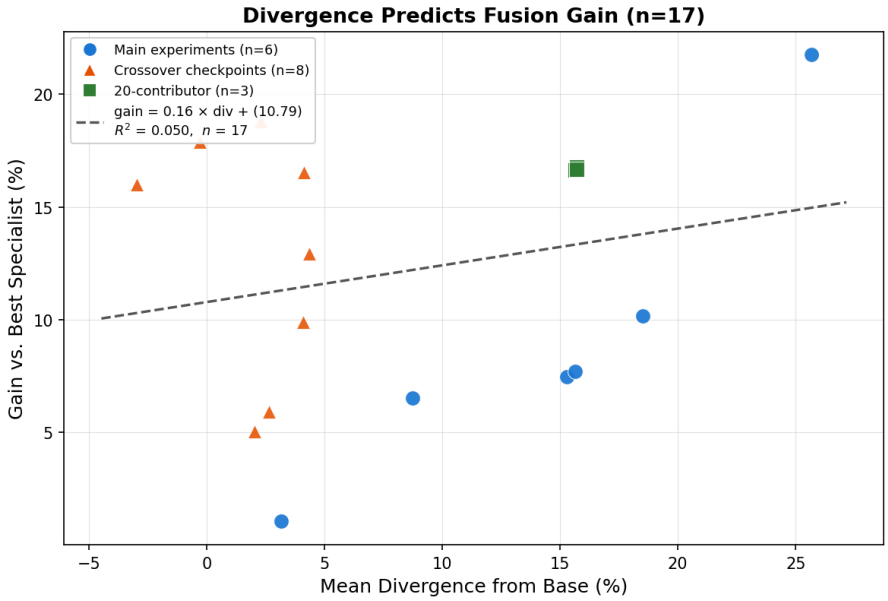

Figure 12: Expanded divergence–gain scatter ($n = 19$). The eight in-sample conditions (circles, including P1/P2 regression extension) are supplemented with eight training-duration crossover conditions (triangles, 50–20,000 steps) and three 20-contributor seeds (squares). The linear law ($R^2 = 0.872$, $n = 8$) holds within a fixed training protocol; across training-duration regimes the relationship is non-monotonic (gain rises then falls as specialists over-train). The 20-contributor points (15.7% divergence, +16.7% gain) lie above the in-sample regression line, consistent with cooperative size amplifying fusion benefit.

R Expanded Divergence–Gain Scatter

S Evaluation Correction Methodology

During development, an initial evaluation protocol produced +14.2% at Pythia-410M. Code review identified two inconsistencies; the corrected protocol yields +7.72%. This appendix documents the bugs and the fix for reproducibility.

Bug A: Asymmetric batch sizes. The original evaluation used batch size 2 for the MoE model and batch size 4 for all baselines (specialist and base). PackedChunkDataset packing means different batch sizes evaluate different token subsequences. Since the MoE was evaluated on different data than its baselines, the comparison was not valid. **Fix**: all models evaluated at batch size 4 (bs=4 across all conditions).

Bug B: Concatenated mixed evaluation. The original evaluation concatenated code, science, and fiction chunks into a single mixed dataset and computed one aggregate loss. Due to chunk ordering, fiction chunks were systematically under-represented in the MoE evaluation pass. Since the MoE had its largest advantage on fiction (the domain with highest specialist divergence, 25.4%), the mixed-batch eval underweighted the domain where MoE gained most vs. the domain where it gained least. **Fix**: evaluate each domain separately at consistent batch size, then compute equal-weight average: $\frac{1}{3}(\mathcal{L}_{\text{code}} + \mathcal{L}_{\text{sci}} + \mathcal{L}_{\text{fiction}})$.

Additional 6.9B fix. The 6.9B result was stabilised by seeded shuffling of the evaluation dataset (original: mean +2.72%, std ±8.17% across seeds; corrected: +6.53% ±0.024% over 3 seeds, computed from stored per-domain losses without re-running specialists). The high variance in the original 6.9B result was caused by non-deterministic chunk ordering producing different effective evaluation sets per seed.

Corrected infrastructure. The corrected evaluation protocol is implemented in `experiments/kalavai_eval_utils.py` (`eval_all_domains`, `eval_loss_domain`). All experiments import this module rather than implementing inline evaluation, preventing recurrence. Original result files were re-evaluated under the corrected protocol.

T Shared Initialisation Ablation

Three conditions test the shared initialisation requirement (Pythia-410M, 2,000 specialist training steps, 3 domains, seed 42 for small-gap and 3 seeds for control/large-gap): (1) *Control*: all specialists from step 10,000 (identical initialisation); (2) *Large gap*: specialists from step 5,000 / 10,000 / 20,000 respectively (spanning $2\times$ training progress); (3) *Small gap*: step 8,000 / 10,000 / 12,000 ($\pm 20\%$ around anchor).

Table 22: Shared initialisation ablation at Pythia-410M. *Note: this experiment uses mixed-domain held-out evaluation, not the per-domain equal-weight protocol used elsewhere in the paper. All values are internally consistent; the meaningful comparison is the relative degradation across conditions, not the absolute loss values.* "MoE loss" is absolute mixed-domain held-out loss; lower is better. "Best Spec. Loss" is the best individual specialist's mixed-domain loss for that condition. "vs. Base" uses base mixed-domain loss 2.248. "vs. Spec" is improvement over best individual specialist—this metric is misleading under mismatch because mismatched specialists are also worse (best spec loss degrades from 2.089 to 2.157 under large gap); the absolute MoE loss is the appropriate comparison.

Condition	Init Revisions	Best Spec. Loss	MoE Loss	vs. Base	Seeds
Control (matched)	10k / 10k / 10k	2.089	**2.015**	**+10.4%**	3
Small gap	8k / 10k / 12k	2.122	2.034	+9.5%	1
Large gap	5k / 10k / 20k	2.157	2.036	+9.4%	3

The absolute MoE quality degrades by 0.93pp (10.37%→9.44%) under the large-gap condition. However, the routing behaviour degrades more substantially: under the large-gap condition, the code expert receives 11% weight on fiction inputs versus near-zero under control. The router can no longer reliably distinguish specialist roles.

Shared initialisation is not strictly required for fusion to produce positive improvement over base—mismatched specialists still produce a positive fused model. However, shared initialisation is important for routing stability: mismatched checkpoints produce routing confusion that may become more pronounced at scale. The shared initialisation requirement is the only coordination cost of the KALAVAI protocol and remains our recommendation for all deployments.

U Heterogeneous Cooperative Robustness

A real cooperative will not have identical training conditions across contributors. We test the protocol's robustness to three practical sources of variation (Pythia-410M, seed 42):

- *Control*: all three specialists trained identically (bs=2, lr=2×10^{-5}, 2,000 steps).
- *diff_batch*: one specialist trained at bs=4 instead of bs=2.
- *diff_lr*: one specialist trained at lr=4×10^{-5} instead of 2×10^{-5}.
- *diff_steps*: one specialist trained for 1,000 steps instead of 2,000.

The maximum spread across all heterogeneous conditions is 0.41pp—well within the noise floor of the experiment. The protocol is robust to realistic variation in batch size, learning rate, and training budget. The only meaningful degradation is a 0.39pp reduction when one specialist trains for half the steps; even then, the fusion gain remains +7.33% vs. best specialist. This validates the cooperative premise: contributors do not need to coordinate hyperparameters, only the shared checkpoint and architecture.

Table 23: Heterogeneous cooperative results (Pythia-410M, per-domain equal-weight evaluation). "Δ vs. control" is the difference in fusion gain from the identical-conditions baseline.

Condition	MoE EW Loss	vs. Spec	vs. Base	Δ vs. control
Control (identical)	2.2185	+7.72%	+16.33%	—
diff_batch	2.2401	+7.74%	+15.49%	+0.01pp
diff_lr	2.1576	+7.73%	+18.63%	+0.01pp
diff_steps	2.2001	+7.33%	+17.00%	−0.39pp

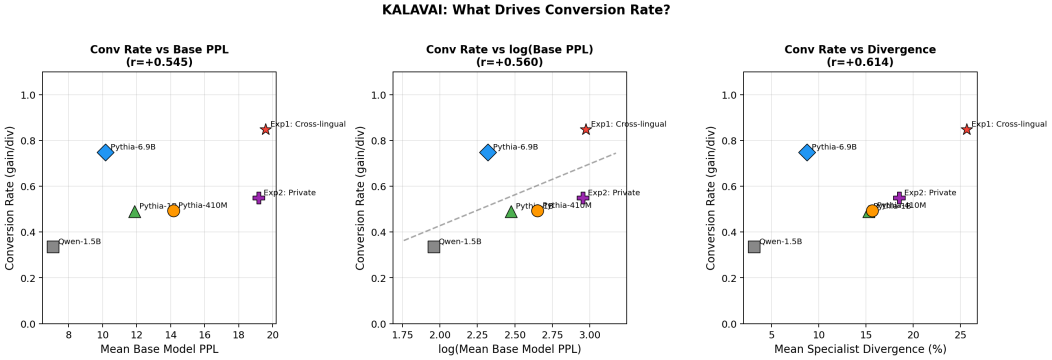

Figure 13: Base-model perplexity as a secondary predictor of cooperative fusion efficiency. **Left**: Conversion efficiency (gain / divergence) versus mean base-model perplexity per condition. **Centre**: Same with log-scaled perplexity axis (Pearson $r = +0.560$). **Right**: Divergence versus gain coloured by base-model PPL quartile. Cross-lingual conditions (high base PPL) convert divergence most efficiently; English-domain conditions (low base PPL) sit near the baseline conversion rate. Dashed lines are OLS fits; $n = 6$ conditions (figure generated on original in-sample set; see main text for updated $n = 8$ regression).

V Base-Model Competence as Secondary Predictor

Specialist divergence captures how much specialists move from the base model; a complementary factor is how competent the base model already is on the target domain. Across the six experimental conditions, the log of the mean base-model perplexity on each domain's evaluation data correlates with the conversion efficiency (gain / divergence) at $r = +0.560$ (Pearson, $n = 6$), compared with $r = +0.614$ for divergence alone. On the six-point sample this is suggestive rather than definitive, but the pattern is mechanistically plausible: when the base model achieves near-random perplexity on a domain (Yoruba PPL ≈ 42, Welsh PPL ≈ 103), the specialist must correct the base from a high-loss floor, the router routes with near-certainty, and essentially all specialist gain is preserved. When the base is already competent (English code PPL ≈ 7), specialist gains are smaller in absolute terms and the cooperative receives less incremental value. This suggests a two-factor heuristic—measure both specialist divergence and base-model competence before committing to a cooperative—though validation on more conditions is needed before treating the secondary predictor as quantitatively reliable. Figure 13 shows the relationship across all conditions.

W Per-Domain Breakdown (410M)

X Divergence–Gain Summary Table

Y Cross-Domain Evaluation Loss Matrix

NeurIPS Paper Checklist

1. **Claims**

Table 24: Per-domain held-out loss at Pythia-410M (seed 42). The KALAVAI MoE matches the best individual specialist on every domain simultaneously.

Method	Code ↓	Science ↓	Fiction ↓	EW Avg ↓
Base model	2.0872	2.8920	2.9739	2.6510
Code specialist	**1.8791**	2.9085	2.9768	2.5881
Science specialist	2.1738	**2.5565**	3.0613	2.5972
Fiction specialist	2.1018	2.8904	2.2194	2.4039
Monolithic (6k)	1.9644	2.6389	**2.0832**	2.2288
Weight averaging	2.0142	2.7323	2.7106	2.4857
KALAVAI MoE	**1.8791**	**2.5565**	2.2194	**2.2183**

Table 25: Summary of divergence–gain relationship across all Phase 1 and Phase 2 experiments. Predicted gain from linear fit $= -2.80 + 0.83 \times$ div; residual = actual−predicted. Predicted values are computed from full-precision OLS coefficients (slope $= 0.8335$, intercept $= -2.7956$). The regression was fit on the eight in-sample conditions (rows 1–8); Exp 3 (row 9) is an out-of-sample validation point. †Qwen predicted gain is at the divergence floor (≈3.3%). ‡In a 20-specialist cooperative the best specialist is only 0.46% above base; nominal conversion rate > 1 reflects this near-equal baseline.

Condition	Mean Div.	Gain vs Spec	Conv. rate	Pred. gain	Residual
Qwen-1.5B (Ph.1)	3.16%	+1.06%	0.34×	≈0%†	—
Pythia-6.9B (Ph.1)	8.73%	+6.53%	0.75×	+4.48%	+2.05pp
P1: 2-domain (regression)	10.77%	+6.22%	0.58×	+6.18%	+0.04pp
Pythia-1B (Ph.1)	15.28%	+7.49%	0.49×	+9.94%	−2.45pp
Pythia-410M (Ph.1)	15.65%	+7.72%	0.49×	+10.25%	−2.53pp
Exp 2: private (Ph.2)	18.52%	+10.17%	0.55×	+12.64%	−2.47pp
P2: 4-domain (regression)	19.84%	+14.71%	0.74×	+13.74%	+0.97pp
Exp 1: cross-lingual (Ph.2)	25.65%	+21.87%	0.85×	+18.58%	+3.29pp
Exp 3: 18-contrib/FE-03 (Ph.2, OOS)	*19.75%*	*+21.13%*	*1.07×*‡	*+13.67%*	*+7.46pp*

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [Yes]

Justification: Every quantitative claim in the abstract and introduction is supported by experimental results in the main paper or appendix. The scope of the divergence–gain regression ($n = 8$, 3–26% divergence, single fixed training protocol) is stated explicitly in both the abstract and Section 4.5, and limitations of the formula are discussed in Section 5.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 5 discusses the main limitations: the $N\times$ inference overhead that scales with cooperative size; the regression formula's empirical basis ($n = 8$ in-sample conditions); router collapse at high divergence under certain random seeds (Section 4.7);

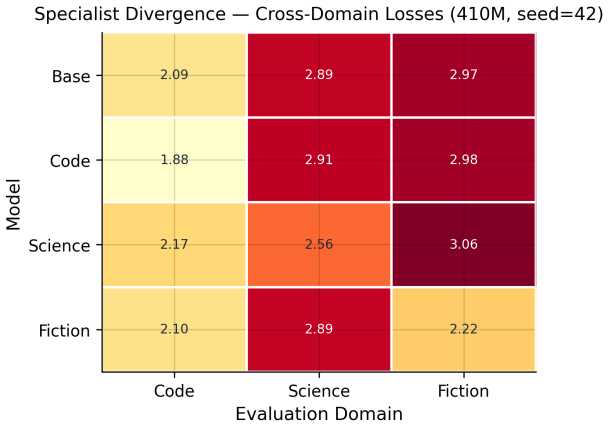

Figure 14: Cross-domain evaluation loss matrix at Pythia-410M, step 2,000 (freeze=4, seed=42). Green = lower loss; red = higher. Diagonal entries confirm each specialist has diverged complementarily; the MoE router recovers diagonal performance across all domains simultaneously.

failure of LoRA-based specialist training to achieve sufficient divergence (Section 3). The 20-contributor experiment identifies dialogue and instructions as domains where fine-tuning on available data actively hurts divergence, and this is noted alongside the result.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper contains no formal theorems or proofs. The divergence–gain relationship is an empirical linear regression (OLS) with slope, intercept, R^2, and 95% CI on the slope reported; no theoretical derivation of the relationship is claimed.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 4.1 specifies the base model checkpoints (Pythia step10000, Qwen-1.5B), domains and datasets, sequence length (512 tokens), batch sizes (bs=4 for evaluation, physical bs=4 with gradient accumulation for router training), learning rates (2×10^{-5} specialist, 2×10^{-4} router), optimizer (AdamW), and freeze configurations tested. The evaluation protocol — per-domain separate evaluation at bs=4 with equal-weight averaging — is described in Section 4.1 and fully documented in Appendix F, including the two bugs in the original protocol and their fixes. Random seeds used for all multi-seed results are listed explicitly (42, 137, 2026).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Anonymised experiment scripts, evaluation utilities, and result JSON artefacts for all reported experiments are provided as supplementary material. All base models (Pythia, Qwen-1.5B) and training datasets (CodeSearchNet, SciQ, PG-19, Wikipedia) are publicly available on HuggingFace Hub. The supplementary code includes a README with environment setup and reproduction instructions; full code will be released publicly upon publication.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 4.1 covers all core hyperparameters. For each domain, 90% of samples are used for specialist training and 10% held out for evaluation; the held-out split is never seen during training or router training. Specialist learning rate (2×10^{-5}) and router learning rate (2×10^{-4}) are as stated in Section 4.1; freeze depth was varied in a dedicated ablation (Section 4.4 and Appendix B); router architecture was ablated in Section 5.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: All main Phase 1 results are reported as means over 3 seeds (42, 137, 2026) with standard deviation in percentage points (e.g., $\pm$0.02pp at 410M, $\pm$0.01pp at 1B, $\pm$0.024pp at 6.9B). Phase 2 experiments provide per-seed tables in Appendix G–H. The regression slope's 95% confidence interval is reported ([0.52, 1.15]).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Pythia-410M and Pythia-1B experiments were run on a single NVIDIA GeForce RTX 5090 (32 GB VRAM); Pythia-6.9B experiments were run on a cloud-hosted NVIDIA A100 (80 GB VRAM). Inference latency and VRAM benchmarks for the RTX 5090 are reported in Appendix I. Training compute is implicitly bounded by the step counts and model sizes given in Section 4.1 (e.g., 50 specialist steps takes under two minutes on the RTX 5090; the main 410M experiments use 2,000 steps per specialist).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: All datasets used are publicly released academic benchmarks. No human subjects were involved and no sensitive personal data was collected or processed.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 5 discusses positive impacts (lowering the compute barrier for language model training, enabling under-resourced language communities to build models collaboratively, supporting privacy-preserving data handling). It also discusses the dual-use risk that the privacy-preserving design introduces: contributors can train specialists on data invisible to any coordinator, which could allow harmful content to enter the fused model without oversight, and that production deployments should incorporate contributor vetting and checkpoint validation.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper describes a training protocol evaluated on standard academic benchmarks using publicly available Pythia-scale models (up to 6.9B parameters). No novel model weights or datasets are released; the method is not specific to high-risk content generation. Special safeguards beyond responsible standard release are not warranted.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All models and datasets are properly cited in the references. Pythia is released under Apache 2.0; Qwen-1.5B under its permissive Qwen License; CodeSearchNet under MIT; SciQ under CC-BY 4.0; PG-19 under Apache 2.0; Wikipedia data under CC-BY-SA 4.0. No dataset or model weights are redistributed.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: Anonymised experiment scripts, evaluation utilities (`kalavai_eval_utils.py`), and result JSON artefacts for all reported experiments are provided as supplementary material with a README covering environment requirements, dataset download steps, and the commands needed to reproduce each main result.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No crowdsourcing or human subjects research was conducted. All experiments involve only language models and publicly available text datasets.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects research was conducted; IRB review was not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are the *subject* of this paper, not a methodological tool. All training and evaluation use standard PyTorch and HuggingFace Transformers infrastructure. Coding assistance tools were used for implementation scripting; per NeurIPS policy, this does not constitute use of LLMs as an original methodological component requiring declaration.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.